

CAPSTONE PROJECT LEVEL 2

Credit Card Spend Summarizer

Course Code: BFSI-CC-003 | Difficulty: Intermediate | Duration: 16 hours

"Your card knows your habits better than you do."

Refer to the Knowledgebase file here.

https://docs.google.com/document/d/15FEVwB1NSVBLr2hwx4gpAyBPYlpA-J7X_iHgVd9qKqU/edit?usp=sharing

PROJECT OVERVIEW

1.1 What is This Project?

A FastAPI service that accepts a credit card account ID and billing month, queries a PostgreSQL transaction database, and uses an LLM to generate a clear, human-readable monthly spend summary — including category breakdown, top merchants, international spend, reward points earned, and a month-over-month comparison.

1.2 Business Problem

- Customers receive raw transaction lists in statements but lack a concise narrative summary of their spending patterns
- Relationship managers need quick spend insights to offer relevant product upsells (e.g., travel card upgrade)
- Fraud and anomaly detection teams need natural language summaries of unusual spend shifts
- Customer service agents waste time manually reading statements to answer spend-related queries

1.3 Sample Queries the System Must Handle

- Summarise my spending for March 2026 on card CC-881001
- What did I spend the most on last month?
- How much did I spend internationally this billing cycle?
- Compare my spending this month vs last month
- How many reward points did I earn this month and what is their value?

- Am I on track to meet the annual fee waiver threshold?

1.4 End Goal

User provides card_id + billing_month → FastAPI handler → SQL queries on Postgres → LLM generates narrative spend summary with category table, merchant highlights, international spend flag, reward summary, and MoM comparison.

SYSTEM ARCHITECTURE

2.1 Architecture Overview

[POST /api/v1/summarise] → [FastAPI Handler] → [SQL Query Engine (Postgres)] → [Context Builder] → [LLM Prompt] → [Structured Spend Summary Response]

2.2 SQL Query Pipeline

Step	Query	Output Used For
1	Fetch all transactions for card + billing period	Base dataset for all computations
2	GROUP BY category_name — sum amounts, count txns	Category breakdown table
3	Filter is_international = TRUE	International spend sub-total
4	ORDER BY amount DESC LIMIT 5	Top 5 merchants
5	SUM reward_pts_earned for the period	Reward points narrative
6	Previous month same aggregation	Month-over-month comparison
7	YTD spend vs fee waiver threshold	Fee waiver alert

2.3 LLM Prompt Design

The LLM receives a structured JSON context (query results) and is instructed to produce a conversational yet precise summary. Key prompt rules:

- Address the customer by first name (from customer record)
- Lead with total spend and the single biggest category
- Call out any international transactions explicitly with the forex markup implication
- State reward points earned and their INR redemption value
- Include a one-line MoM comparison (up/down %, or "first month on record")
- End with a relevant tip or product suggestion based on spend pattern

- Never fabricate numbers — only use values from the SQL context provided

COURSE MODULE MAPPING

- Module 2.1–2.4: Python basics, SQL queries, PostgreSQL connection (psycopg2 / SQLAlchemy)
- Module 3.1: Schema understanding — card_transactions, billing_statements, reward_transactions
- Module 3.2: Writing aggregation queries — GROUP BY, FILTER, window functions for MoM
- Module 3.3: LLM integration — building context JSON, prompt template, parsing response
- Module 3.4: FastAPI endpoint — request validation, async DB calls, response model
- Module 4.1: Basic evaluation — LLM output vs. SQL ground truth numbers

TECHNOLOGY STACK

Component	Technology
Language	Python 3.10+
LLM	Google Gemini / OpenAI GPT-4o-mini
RDBMS	PostgreSQL 16+
ORM / DB Driver	SQLAlchemy 2.0+ or psycopg2
API Framework	FastAPI 0.100+
Async Support	asyncio, asyncpg (optional)
Monitoring	LangSmith (optional for this level)
Testing	pytest + httpx

API SPECIFICATIONS

Endpoint: POST /api/v1/summarise or /query

Request:

```
{ "card_id": "CC-881001", "billing_month": "2026-03" }
```

Response (200 OK):

Field	Description
card_id	Card identifier
customer_name	First name of cardholder
billing_month	e.g. "March 2026"
total_spend	Total purchase amount for the month
total_transactions	Count of purchase transactions

category_breakdown	Array of {category, amount, count, pct_of_total}
top_merchants	Array of top 5 {merchant_name, amount}
international_spend	Total international spend + transaction count
reward_points_earned	Points earned + INR value
mom_change_pct	Month-over-month spend change as percentage
summary_text	LLM-generated narrative summary (2–4 sentences)
tip	LLM-generated personalised tip or product suggestion

EVALUATION CRITERIA & GRADING RUBRIC

Component	Weight	Details
SQL Query Accuracy	35%	All 7 queries produce correct results vs. seed data
LLM Summary Quality	25%	Numbers match SQL output; clear narrative; no hallucination
API & Integration	20%	FastAPI endpoint works; correct request/response schema; error handling
Code Quality	15%	Modular structure, docstrings, basic test coverage
Bonus: Fee Waiver Alert	+5%	Correctly computes and flags fee waiver gap in summary

Key Success Metrics

- SQL accuracy: 100% of computed numbers match ground truth in seed data
- LLM grounding: 0 hallucinated figures in spend summary
- API response time: <800ms (p95) on local Postgres
- Category coverage: All 15 spend categories handled without error

DELIVERABLES

- Python source code (src/ folder — db.py, queries.py, summariser.py, main.py)
- FastAPI app with Swagger docs (/docs)

- PostgreSQL schema + seed_data_cc.sql (provided — see Appendix)
- At least 3 sample API responses in a responses/ folder (JSON files)
- README with setup instructions and example curl command

APPENDIX: SQL SEED DATA

The following seed data creates all tables and populates them with realistic credit card transaction data for NorthStar Bank. The primary demo account is CC-881001 (James Mitchell / Gold Card) with 3 months of transaction history. Run this against PostgreSQL 16+ before starting the project.

Database Tables Overview

Table	Rows	Description
customers	6	Cardholder personal details
credit_cards	6	Card variants, limits, outstanding, reward points
card_transactions	47	Purchase, payment, refund, fee transactions (3 months)
reward_transactions	5	Points earned, redeemed, and expired per billing cycle
billing_statements	4	Pre-computed monthly statement summaries

Key Accounts for Testing

Card ID	Customer	Variant	Credit Limit	Primary Use
CC-881001	James Mitchell	Gold	₹2,00,000	Main demo — 3 months data, intl txns
CC-882001	Sarah Thompson	Platinum	₹5,00,000	High-spend, international travel
CC-883001	Daniel Foster	Classic	₹75,000	Low spend, fee waiver scenario
CC-884001	Robert Clarke	Signature	₹10,00,000	Ultra-HNI profile
CC-885001	Emily Watson	Gold	₹1,50,000	Routine monthly spender
CC-886001	Laura Bennett	Classic	₹50,000	Minimal transactions

Sample NL-to-SQL Test Cases

Q1 — Category Breakdown

NL Query: Summarise my spending by category for March 2026 on CC-881001

SQL:

```

SELECT category_name, COUNT(*) AS txn_count,
       SUM(amount) AS total_spend,
       SUM(reward_pts_earned) AS points
FROM card_transactions
WHERE card_id = 'CC-881001'
      AND txn_date BETWEEN '2026-02-26' AND '2026-03-25'
      AND txn_type = 'purchase'
GROUP BY category_name ORDER BY total_spend DESC;

```

Q2 — International Spend

NL Query: Show international transactions on CC-881001

SQL:

```

SELECT txn_date, merchant_name, amount,
       original_currency, original_amount
FROM card_transactions
WHERE card_id = 'CC-881001'
      AND is_international = TRUE
ORDER BY txn_date DESC;

```

Q3 — Month-over-Month

NL Query: Compare spending this month vs last month for CC-881001

SQL:

```

SELECT TO_CHAR(txn_date, 'YYYY-MM') AS month,
       SUM(amount) FILTER (WHERE txn_type='purchase') AS total
FROM card_transactions
WHERE card_id = 'CC-881001'
GROUP BY 1 ORDER BY 1;

```

Q4 — Fee Waiver Gap

NL Query: Is CC-883001 on track for annual fee waiver?

SQL:

```

SELECT SUM(amount) AS ytd_spend,
       50000 AS target,
       GREATEST(0, 50000 - SUM(amount)) AS remaining
FROM card_transactions
WHERE card_id = 'CC-883001'
      AND txn_type = 'purchase'
      AND EXTRACT(YEAR FROM txn_date) = 2026;

```

Q5 — Reward Points Value

NL Query: How many reward points has James earned on CC-881001?

SQL:

```
SELECT reward_points,
       reward_points * 0.25 AS inr_value
FROM credit_cards
WHERE card_id = 'CC-881001';
```

Complete seed_data_cc.sql

The full SQL script is embedded below. It includes CREATE TABLE statements for all 5 tables, INSERT data for 6 customers, 6 cards, 47 transactions across 3 billing months, and 6 commented sample queries for NL-to-SQL node validation.

```
-- =====
-- NorthStar Bank – Credit Card Spend Summarizer
-- Seed Data | Capstone Project BFSI-CC-003
-- PostgreSQL 16+
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- =====
-- 1. SCHEMA
-- =====

CREATE TABLE IF NOT EXISTS customers (
  customer_id    VARCHAR(20) PRIMARY KEY,
  full_name      VARCHAR(100) NOT NULL,
  email          VARCHAR(100),
  mobile         VARCHAR(15),
  dob            DATE,
  kyc_status     VARCHAR(20) DEFAULT 'verified',
  created_at     TIMESTAMP   DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS credit_cards (
  card_id        VARCHAR(20) PRIMARY KEY,
  customer_id    VARCHAR(20) REFERENCES customers(customer_id),
  card_variant   VARCHAR(30) NOT NULL,
  credit_limit   NUMERIC(15,2),
  available_limit NUMERIC(15,2),
  cash_limit     NUMERIC(15,2),
  outstanding_amt NUMERIC(15,2) DEFAULT 0,
  statement_date INT,           -- day of month e.g. 25
  due_date       INT,           -- day of month e.g. 15 (next month)
  min_due        NUMERIC(15,2) DEFAULT 0,
  reward_points  INT            DEFAULT 0,
  status         VARCHAR(20) DEFAULT 'active',
  issued_date    DATE,
  created_at     TIMESTAMP   DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS card_transactions (
  txn_id         UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  card_id        VARCHAR(20) REFERENCES credit_cards(card_id),
  txn_date       DATE NOT NULL,
  posting_date   DATE,
  txn_type       VARCHAR(20) NOT NULL CHECK (txn_type IN
('purchase','cashadvance','payment','refund','fee','emi_instalment')),
  amount         NUMERIC(15,2) NOT NULL,
  original_currency VARCHAR(5) DEFAULT 'INR',
  original_amount NUMERIC(15,2),
  merchant_name  VARCHAR(100),
```



```

        category_code    VARCHAR(10),
        category_name    VARCHAR(50),
        is_international  BOOLEAN      DEFAULT FALSE,
        is_emi            BOOLEAN      DEFAULT FALSE,
        emi_months        INT,
        reward_pts_earned INT          DEFAULT 0,
        status            VARCHAR(20)   DEFAULT 'posted', -- posted / disputed / reversed
        created_at        TIMESTAMP     DEFAULT NOW()
    );

CREATE TABLE IF NOT EXISTS reward_transactions (
    reward_txn_id    UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    card_id          VARCHAR(20)       REFERENCES credit_cards(card_id),
    txn_date         DATE,
    points_earned    INT               DEFAULT 0,
    points_redeemed  INT               DEFAULT 0,
    points_expired   INT               DEFAULT 0,
    description      VARCHAR(200),
    expiry_date      DATE,
    created_at       TIMESTAMP         DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS billing_statements (
    statement_id     UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    card_id          VARCHAR(20)       REFERENCES credit_cards(card_id),
    billing_month     VARCHAR(10),      -- e.g. '2026-03'
    start_date        DATE,
    end_date          DATE,
    due_date          DATE,
    opening_balance   NUMERIC(15,2),
    total_purchases   NUMERIC(15,2),
    total_payments    NUMERIC(15,2),
    total_fees        NUMERIC(15,2),
    total_refunds     NUMERIC(15,2),
    closing_balance   NUMERIC(15,2),
    min_amount_due    NUMERIC(15,2),
    reward_pts_earned INT,
    generated_at      TIMESTAMP         DEFAULT NOW()
);

-- =====
-- 2. CUSTOMERS
-- =====
INSERT INTO customers (customer_id, full_name, email, mobile, dob, kyc_status) VALUES
('C-1001', 'James Mitchell', 'james.mitchell@email.com', 'XXXXXXXX890',
'1985-06-14', 'verified'),
('C-1002', 'Sarah Thompson', 'sarah.thompson@email.com', 'XXXXXXXX123',
'1990-03-22', 'verified'),
('C-1003', 'Robert Clarke', 'robert.clarke@bizmail.com', 'XXXXXXXX456',
'1978-11-05', 'verified'),
('C-1004', 'Emily Watson', 'emily.watson@email.com', 'XXXXXXXX789',
'1995-07-30', 'verified'),
('C-1005', 'Daniel Foster', 'daniel.foster@email.com', 'XXXXXXXX321',
'1988-01-18', 'verified'),
('C-1006', 'Laura Bennett', 'laura.bennett@email.com', 'XXXXXXXX654',
'1992-09-09', 'verified');

-- =====
-- 3. CREDIT CARDS
-- =====
INSERT INTO credit_cards (card_id, customer_id, card_variant, credit_limit,
available_limit, cash_limit, outstanding_amt, statement_date, due_date, min_due,
reward_points, issued_date) VALUES
('CC-881001', 'C-1001', 'NorthStar Gold', 200000, 145000, 40000, 55000, 25, 15,
2750, 3420, '2021-07-01'),
('CC-882001', 'C-1002', 'NorthStar Platinum', 500000, 420000, 100000, 80000, 25, 15,
4000, 8760, '2022-03-15'),
('CC-883001', 'C-1005', 'NorthStar Classic', 75000, 60000, 15000, 15000, 20, 10,
750, 640, '2023-01-10'),
('CC-884001', 'C-1003', 'NorthStar Signature', 1000000, 850000, 200000, 150000, 28, 18,
7500, 22100, '2024-02-01'),

```

```
('CC-885001', 'C-1004', 'NorthStar Gold',      150000, 132000,  30000, 18000, 22, 12,
900,  1200, '2023-08-20'),
('CC-886001', 'C-1006', 'NorthStar Classic',    50000,  45000,  10000,  5000, 15,  5,
250,   310, '2024-06-01');
```

```
--
-- 4. CARD TRANSACTIONS - CC-881001 (James Mitchell / Gold)
-- Primary account for Spend Summarizer demos
--
```

```
-- January 2026 (billing cycle Jan 26 - Feb 25)
INSERT INTO card_transactions (card_id, txn_date, posting_date, txn_type, amount,
original_currency, original_amount, merchant_name, category_code, category_name,
is_international, reward_pts_earned) VALUES
('CC-881001', '2026-01-27', '2026-01-27', 'purchase',  3200.00, 'INR',  3200.00, 'Barbeque
Nation',      'FOOD', 'Food & Dining',  FALSE, 64),
('CC-881001', '2026-01-29', '2026-01-29', 'purchase',  4500.00, 'INR',  4500.00, 'BigBasket',
'GROC', 'Groceries',      FALSE, 45),
('CC-881001', '2026-01-31', '2026-01-31', 'purchase',  1200.00, 'INR',  1200.00, 'Spotify
Premium',    'ENTR', 'Entertainment',  FALSE, 12),
('CC-881001', '2026-02-02', '2026-02-02', 'purchase', 12000.00, 'INR', 12000.00, 'Myntra',
'SHOP', 'Shopping',      FALSE, 120),
('CC-881001', '2026-02-05', '2026-02-05', 'purchase',  2100.00, 'INR',  2100.00, 'Dominos
Pizza',      'FOOD', 'Food & Dining',  FALSE, 42),
('CC-881001', '2026-02-08', '2026-02-08', 'purchase',  3500.00, 'INR',  3500.00, 'BESCOM
Electricity', 'UTIL', 'Utilities',      FALSE, 35),
('CC-881001', '2026-02-10', '2026-02-10', 'purchase', 28000.00, 'INR', 28000.00, 'IRCTC Train
Tickets',    'TRVL', 'Travel',          FALSE, 560),
('CC-881001', '2026-02-12', '2026-02-12', 'purchase',  6700.00, 'INR',  6700.00, 'Reliance
Digital',    'ELEC', 'Electronics',     FALSE, 67),
('CC-881001', '2026-02-14', '2026-02-14', 'purchase',  3800.00, 'INR',  3800.00, 'Zomato',
'FOOD', 'Food & Dining',  FALSE, 76),
('CC-881001', '2026-02-15', '2026-02-17', 'purchase',  9800.00, 'INR',  9800.00, 'Decathlon',
'SHOP', 'Shopping',      FALSE, 98),
('CC-881001', '2026-02-18', '2026-02-18', 'purchase',  1800.00, 'INR',  1800.00, 'Netflix',
'ENTR', 'Entertainment',  FALSE, 18),
('CC-881001', '2026-02-20', '2026-02-20', 'purchase',  4200.00, 'INR',  4200.00, 'Apollo
Pharmacy',   'HLTH', 'Health & Medical', FALSE, 42),
('CC-881001', '2026-02-22', '2026-02-22', 'purchase', 15000.00, 'INR', 15000.00, 'MakeMyTrip
Hotels',     'TRVL', 'Travel',          FALSE, 300),
('CC-881001', '2026-02-24', '2026-02-24', 'payment', 40000.00, 'INR', 40000.00, 'NorthStar
Payment',    'OTHR', 'Payment',          FALSE,  0),
('CC-881001', '2026-02-25', '2026-02-25', 'fee',      999.00, 'INR',   999.00, 'NorthStar
Annual Fee', 'OTHR', 'Fee',              FALSE,  0);
```

```
-- March 2026 billing cycle (Feb 26 - Mar 25) - PRIMARY DEMO MONTH
INSERT INTO card_transactions (card_id, txn_date, posting_date, txn_type, amount,
original_currency, original_amount, merchant_name, category_code, category_name,
is_international, reward_pts_earned) VALUES
('CC-881001', '2026-02-27', '2026-02-27', 'purchase',  3200.00, 'INR',  3200.00, 'Barbeque
Nation',      'FOOD', 'Food & Dining',  FALSE, 64),
('CC-881001', '2026-03-02', '2026-03-02', 'purchase', 12000.00, 'INR', 12000.00, 'Myntra',
'SHOP', 'Shopping',      FALSE, 120),
('CC-881001', '2026-03-05', '2026-03-05', 'purchase',  8500.00, 'INR',  8500.00, 'Marriott
Hotels Pune', 'TRVL', 'Travel',          FALSE, 170),
('CC-881001', '2026-03-08', '2026-03-08', 'purchase',  4100.00, 'INR',  4100.00, 'Swiggy',
'FOOD', 'Food & Dining',  FALSE, 82),
('CC-881001', '2026-03-10', '2026-03-10', 'purchase',  2100.00, 'INR',
2100.00, 'BookMyShow',    'ENTR', 'Entertainment',  FALSE, 21),
('CC-881001', '2026-03-12', '2026-03-12', 'purchase', 18500.00, 'INR', 18500.00, 'Amazon UK',
'SHOP', 'Shopping',      TRUE, 185),
('CC-881001', '2026-03-14', '2026-03-14', 'purchase', 32400.00, 'SGD',   480.00, 'Singapore
Airlines',   'TRVL', 'Travel',          TRUE, 648),
('CC-881001', '2026-03-15', '2026-03-15', 'purchase',  3500.00, 'INR',  3500.00, 'BESCOM
Electricity', 'UTIL', 'Utilities',      FALSE, 35),
('CC-881001', '2026-03-17', '2026-03-17', 'purchase',  1500.00, 'INR',  1500.00, 'Spotify
Premium',    'ENTR', 'Entertainment',  FALSE, 15),
('CC-881001', '2026-03-18', '2026-03-18', 'purchase',  9800.00, 'INR',  9800.00, 'Tanishq
Jewellery',  'JEWL', 'Jewellery',        FALSE, 98),
('CC-881001', '2026-03-20', '2026-03-20', 'purchase',  4500.00, 'INR',  4500.00, 'BigBasket',
'GROC', 'Groceries',      FALSE, 45),
```

```

('CC-881001','2026-03-22','2026-03-22','refund', -2000.00,'INR',-2000.00,'Amazon UK
Refund', 'SHOP','Shopping', TRUE, -20),
('CC-881001','2026-03-23','2026-03-23','purchase', 6700.00,'INR', 6700.00,'Croma
Electronics', 'ELEC','Electronics', FALSE, 67),
('CC-881001','2026-03-24','2026-03-24','fee', 340.00,'INR', 340.00,'Forex
Markup Fee', 'OTHR','Fee', FALSE, 0),
('CC-881001','2026-03-25','2026-03-25','fee', 87.50,'INR', 87.50,'GST on
Forex Fee', 'OTHR','Fee', FALSE, 0);

```

-- April 2026 partial month

```

INSERT INTO card_transactions (card_id, txn_date, posting_date, txn_type, amount,
original_currency, original_amount, merchant_name, category_code, category_name,
is_international, reward_pts_earned) VALUES
('CC-881001','2026-03-28','2026-03-28','payment', 55000.00,'INR',55000.00,'NorthStar
UPI Payment', 'OTHR','Payment', FALSE, 0),
('CC-881001','2026-04-01','2026-04-01','purchase', 3100.00,'INR', 3100.00,'Zomato',
'FOOD','Food & Dining', FALSE, 62),
('CC-881001','2026-04-03','2026-04-03','purchase', 6700.00,'INR', 6700.00,'Reliance
Digital', 'ELEC','Electronics', FALSE, 67),
('CC-881001','2026-04-07','2026-04-07','purchase', 2100.00,'INR', 2100.00,'Dominos
Pizza', 'FOOD','Food & Dining', FALSE, 42),
('CC-881001','2026-04-10','2026-04-10','purchase', 18000.00,'INR',18000.00,'IRCTC
Tatkal Tickets', 'TRVL','Travel', FALSE, 360);

```

-- 5. TRANSACTIONS – OTHER CARDS (supporting data)

```

INSERT INTO card_transactions (card_id, txn_date, posting_date, txn_type, amount,
original_currency, merchant_name, category_code, category_name, is_international,
reward_pts_earned) VALUES
-- CC-882001 Sarah Thompson / Platinum
('CC-882001','2026-03-03','2026-03-03','purchase', 5200.00,'INR','Spencer Retail',
'GROC','Groceries', FALSE, 156),
('CC-882001','2026-03-07','2026-03-07','purchase', 15000.00,'INR','Flipkart',
'SHOP','Shopping', FALSE, 450),
('CC-882001','2026-03-12','2026-03-12','purchase', 42000.00,'USD','Hilton New York',
'TRVL','Travel', TRUE, 2520),
('CC-882001','2026-03-15','2026-03-15','purchase', 4800.00,'INR','Zomato Gold Dining',
'FOOD','Food & Dining', FALSE, 288),
('CC-882001','2026-03-20','2026-03-20','purchase', 9800.00,'INR','Nykaa',
'SHOP','Shopping', FALSE, 294),
('CC-882001','2026-04-01','2026-04-01','payment', 80000.00,'INR','NorthStar NACH
Debit', 'OTHR','Payment', FALSE, 0),
-- CC-883001 Daniel Foster / Classic
('CC-883001','2026-03-05','2026-03-05','purchase', 2500.00,'INR','D-Mart',
'GROC','Groceries', FALSE, 25),
('CC-883001','2026-03-10','2026-03-10','purchase', 4200.00,'INR','Amazon India',
'SHOP','Shopping', FALSE, 42),
('CC-883001','2026-03-15','2026-03-15','purchase', 1800.00,'INR','BookMyShow',
'ENTR','Entertainment', FALSE, 18),
('CC-883001','2026-03-20','2026-03-20','purchase', 3200.00,'INR','Swiggy',
'FOOD','Food & Dining', FALSE, 32),
('CC-883001','2026-03-25','2026-03-25','purchase', 3500.00,'INR','Airtel Recharge',
'UTIL','Utilities', FALSE, 35),
('CC-883001','2026-04-02','2026-04-02','payment', 15000.00,'INR','NorthStar UPI',
'OTHR','Payment', FALSE, 0);

```

-- 6. REWARD TRANSACTIONS

```

INSERT INTO reward_transactions (card_id, txn_date, points_earned, points_redeemed,
points_expired, description, expiry_date) VALUES
('CC-881001','2026-02-25', 1332, 0, 0, 'Points earned - Feb billing cycle',
'2027-12-31'),
('CC-881001','2026-03-10', 0, 1000, 0, 'Redemption - Swiggy voucher ₹250',
'2027-12-31'),
('CC-881001','2026-03-25', 1550, 0, 0, 'Points earned - Mar billing cycle',
'2027-12-31'),
('CC-882001','2026-03-25', 3708, 0, 0, 'Points earned - Mar billing cycle',
'2027-12-31'),
('CC-883001','2026-03-20', 152, 0, 0, 'Points earned - Mar billing cycle',
'2027-12-31');

```

```
'2027-12-31');
```

```
-- 7. BILLING STATEMENTS
```

```
INSERT INTO billing_statements (card_id, billing_month, start_date, end_date, due_date,
opening_balance, total_purchases, total_payments, total_fees, total_refunds,
closing_balance, min_amount_due, reward_pts_earned) VALUES
('CC-881001','2026-02','2026-01-26','2026-02-25','2026-03-15', 15000, 96800, 40000,
999, 0, 72799, 3640, 1332),
('CC-881001','2026-03','2026-02-26','2026-03-25','2026-04-15', 72799,105400, 55000,
427.50,2000,121627, 6081, 1550),
('CC-882001','2026-03','2026-02-26','2026-03-25','2026-04-15', 20000, 76800, 80000,
0, 0, 16800, 840, 3708),
('CC-883001','2026-03','2026-02-21','2026-03-20','2026-04-10', 5000, 15200, 15000,
0, 0, 5200, 260, 152);
```

```
-- 8. USEFUL SAMPLE QUERIES (for NL-to-SQL node testing)
```

```
-- Q1: Monthly spend summary by category for CC-881001 (March 2026)
-- SELECT category_name,
--        COUNT(*) FILTER (WHERE txn_type = 'purchase') AS txn_count,
--        SUM(amount) FILTER (WHERE txn_type = 'purchase') AS total_spend,
--        SUM(reward_pts_earned) AS points_earned
-- FROM card_transactions
-- WHERE card_id = 'CC-881001'
--    AND txn_date BETWEEN '2026-02-26' AND '2026-03-25'
--    AND txn_type IN ('purchase')
-- GROUP BY category_name ORDER BY total_spend DESC;

-- Q2: Top 5 merchants by spend for CC-881001 in March billing cycle
-- SELECT merchant_name, SUM(amount) AS total, COUNT(*) AS txns
-- FROM card_transactions
-- WHERE card_id = 'CC-881001'
--    AND txn_date BETWEEN '2026-02-26' AND '2026-03-25'
--    AND txn_type = 'purchase'
-- GROUP BY merchant_name ORDER BY total DESC LIMIT 5;

-- Q3: International transactions for CC-881001
-- SELECT txn_date, merchant_name, amount, original_currency, original_amount,
category_name
-- FROM card_transactions
-- WHERE card_id = 'CC-881001' AND is_international = TRUE
-- ORDER BY txn_date DESC;

-- Q4: Reward points balance for a customer
-- SELECT cc.card_id, cc.card_variant, cc.reward_points,
--        cc.reward_points * 0.25 AS redemption_value_inr
-- FROM credit_cards cc
-- JOIN customers c ON cc.customer_id = c.customer_id
-- WHERE c.customer_id = 'C-1001';

-- Q5: Year-to-date spend vs fee waiver threshold for CC-883001
-- SELECT cc.card_id, cc.card_variant,
--        SUM(ct.amount) AS ytd_spend,
--        CASE cc.card_variant
--            WHEN 'NorthStar Classic' THEN 50000
--            WHEN 'NorthStar Gold' THEN 100000
--            WHEN 'NorthStar Platinum' THEN 300000
--            WHEN 'NorthStar Signature' THEN 700000
--        END AS fee_waiver_target,
--        CASE cc.card_variant
--            WHEN 'NorthStar Classic' THEN GREATEST(0, 50000 - SUM(ct.amount))
--            WHEN 'NorthStar Gold' THEN GREATEST(0, 100000 - SUM(ct.amount))
--            WHEN 'NorthStar Platinum' THEN GREATEST(0, 300000 - SUM(ct.amount))
--            WHEN 'NorthStar Signature' THEN GREATEST(0, 700000 - SUM(ct.amount))
--        END AS remaining_to_waiver
-- FROM credit_cards cc
-- JOIN card_transactions ct ON cc.card_id = ct.card_id
```

```

-- WHERE cc.card_id = 'CC-883001'
--       AND ct.txn_type = 'purchase'
--       AND EXTRACT(YEAR FROM ct.txn_date) = 2026
-- GROUP BY cc.card_id, cc.card_variant;

-- Q6: Month-over-month spend comparison for CC-881001
-- SELECT
--     TO_CHAR(txn_date, 'YYYY-MM') AS month,
--     SUM(amount) FILTER (WHERE txn_type = 'purchase') AS total_purchases,
--     COUNT(*) FILTER (WHERE txn_type = 'purchase') AS txn_count
-- FROM card_transactions
-- WHERE card_id = 'CC-881001'
-- GROUP BY TO_CHAR(txn_date, 'YYYY-MM')
-- ORDER BY month;

```

Project 3: Credit Card Spend Summarizer | BFSI-CC-003 | April 2026